

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Компьютерного проектирования
Кафедра Инженерной психологии и эргономики
Дисциплина Интерфейсы информационных систем

Лабораторная работа №2
«Реализация функциональных UI-компонентов на Vue с JSX и локальным состоянием»

Студент группы 310102

(подпись)

Кадол А.П.

Руководитель

(подпись)

Прудник А.М.

Минск 2026

ВВЕДЕНИЕ

Целью работы является получение практических навыков по созданию веб-приложений на *Vue* с использованием компонентной архитектуры, синтаксиса *JSX* и управления локальным состоянием. Работа направлена на закрепление использования хука `useState` для интерактивных элементов и на освоение передачи данных через *props*.

Для достижения цели поставлены следующие задачи:

- реализовать ключевые *UI*-элементы блогговой платформы «Folio» как функциональные *Vue*-компоненты;
- добавить кнопке лайка локальный счётчик, увеличивающийся по клику;
- хранить текст комментария в локальном состоянии поля ввода;
- передать данные карточки статьи через *props*;
- обеспечить запуск проекта командой `npm run dev` без ошибок.

Результат работы – одностраничное приложение на *Vue 3* и *Vite*, исходный код в *Git*-репозитории и настоящее описание компонентов.

1 ПОСТАНОВКА ЗАДАЧИ И ОГРАНИЧЕНИЯ

По варианту реализована блогговая платформа *Folio*: лента статей, реакции и обсуждение. Визуальная основа – *UI*-кит предыдущей работы (кнопки *Post / Like / Comment*, карточка статьи, поле комментария, аватар). В данной работе те же элементы собраны в коде, а не в макете.

Инструменты: *Vue.js*, *Vite*, *npm*. Разметка компонентов выполнена на *JSX* (плагин `@vitejs/plugin-vue-jsx`). Классовые компоненты не применялись. Внешние библиотеки состояния (*Redux*, *MobX*, *Zustand*) не подключались: вся работа со состоянием ведётся через хук `useState`.

Сводка инструментов – в таблице 1.1.

Таблица 1.1 – Инструменты, использованные в работе

Задача	Инструмент	Назначение
Сборка и <i>dev</i> -сервер	<i>Vite</i>	Команда <code>npm run dev</code> , горячая перезагрузка
Компоненты и <i>JSX</i>	<i>Vue 3</i> , плагин <code>vue-jsx</code>	Функциональные компоненты без файлов <code>.vue</code>
Локальное состояние	хук <code>useState</code>	Счётчик лайков и текст комментария
Графика	<i>SVG</i> -иконки и <i>JPEG</i> -превью из ассетов <i>Folio</i>	Соответствие макету карточки и кнопок

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ И ХУК USESTATE

Приложение – одна страница: шапка, лента карточек и блок обсуждения. Точка входа – `src/main.jsx`, корневой компонент – `src/App.jsx`. Данные двух статей лежат в `src/data/articles.js` и передаются в `ArticleCard` как *props*.

Хук `useState` реализован в `src/hooks/useState.js` поверх реактивной ссылки `Vue(ref)`. Интерфейс совпадает с привычным для функциональных компонентов: функция возвращает пару «значение / сеттер», сеттер принимает либо новое значение, либо функцию от предыдущего. Компоненты не импортируют `ref` напрямую и не используют внешние хранилища.

Каждый экземпляр `LikeButton` и `CommentInput` хранит своё состояние независимо: клик по лайку на первой карточке не меняет счётчик второй.

3 РЕАЛИЗОВАННЫЕ КОМПОНЕНТЫ

Состав компонентов, их *props* и локальное состояние приведены в таблице 3.1. Все компоненты объявлены через `defineComponent` и функцию `setup`, возвращающую *JSX*-разметку.

Таблица 3.1 – Компоненты, *props* и локальное состояние

Компонент	Назначение	<i>Props</i>	Состояние <i>useState</i>
LikeButton	кнопка лайка со счётчиком	<code>initialCount</code>	<code>likes</code> : число, +1 по клику
CommentInput	поле комментария и кнопка <i>Post</i>	<code>placeholder</code>	<code>text</code> , <code>focused</code> , список сообщений
ArticleCard	карточка статьи	<code>title</code> , <code>excerpt</code> , <code>author</code> , <code>date</code> , <code>readTime</code> , <code>image</code> , <code>imageAlt</code> , <code>likes</code> , <code>comments</code>	нет: данные только из <i>props</i>
Avatar	круглая заглушка аватара	<code>size</code> (32 / 48 / 64), <code>src</code> , <code>alt</code>	нет
PostButton	кнопка отправки	<code>disabled</code> , <code>label</code>	нет

`LikeButton` инициализирует счётчик значением `initialCount` (для демонстрационных карточек – 128 и 86). Каждый клик вызывает сеттер `setLikes((count) => count + 1)`. После первого увеличения иконка меняется на залитое сердце, кнопка получает класс `is-liked`.

`CommentInput` хранит строку поля в `text`. Пока строка пустая, кнопка *Post* неактивна (состояние *Disabled* из макета). В фокусе у контейнера появляется обводка цвета бренда. Отправка по кнопке или по *Enter* добавляет текст в локальный список и очищает поле.

`ArticleCard` не хранит заголовок, лид и метаданные у себя: родитель передаёт их из массива статей. Внутри карточки монтируется `LikeButton` с `initialCount={props.likes}`, поэтому интерактив лайка остаётся локальным, а «паспорт» материала – снаружи.

Внешний вид ленты показан на рисунке 3.1. После двух кликов по лайку первой карточки счётчик стал 130 (рисунок 3.2). Поле комментария в фокусе – на рисунке 3.3. После отправки текст появляется под полем, само поле снова пустое (рисунок 3.4).

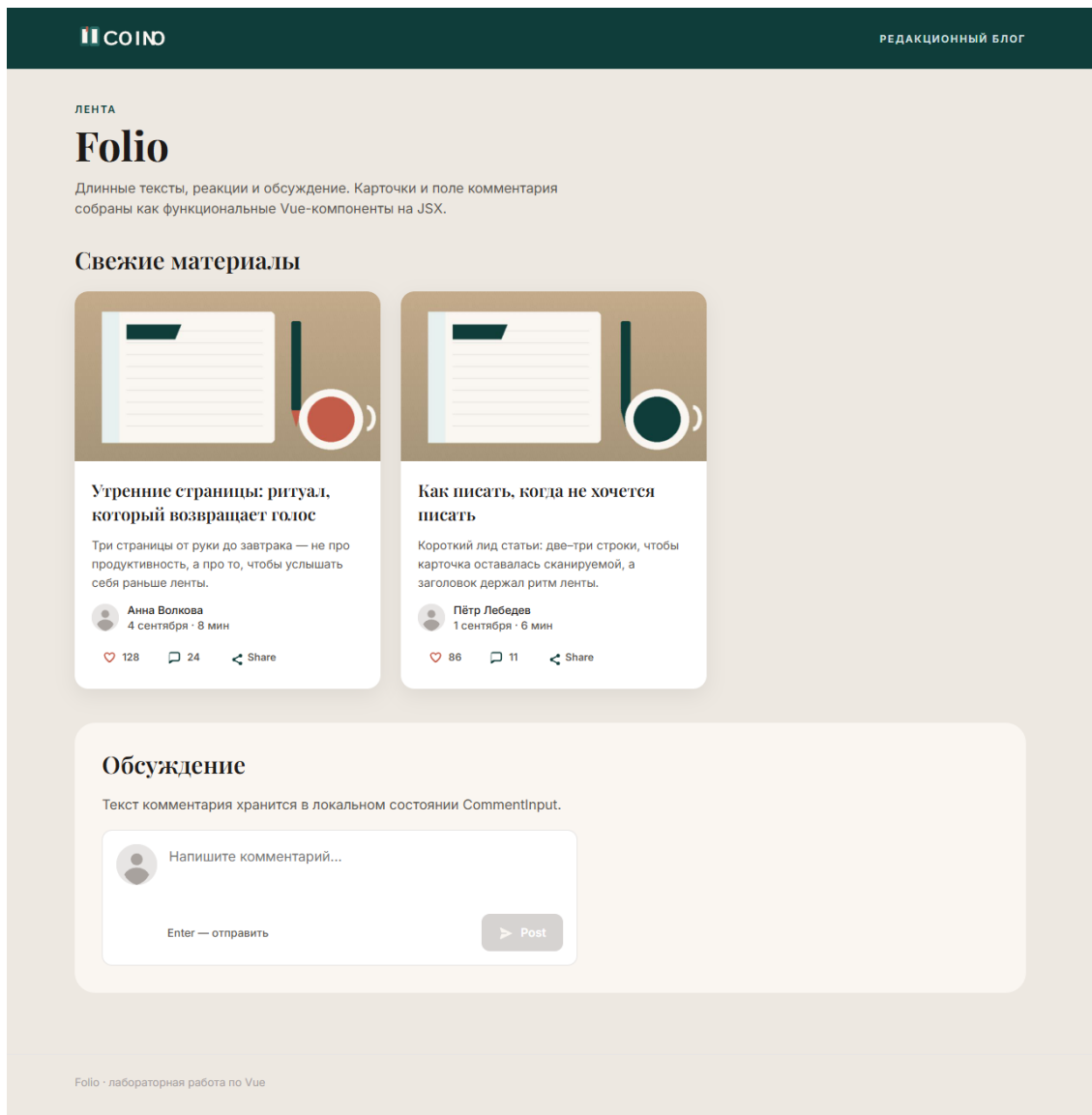


Рисунок 3.1 – Лента Folio: две карточки статьи и блок обсуждения

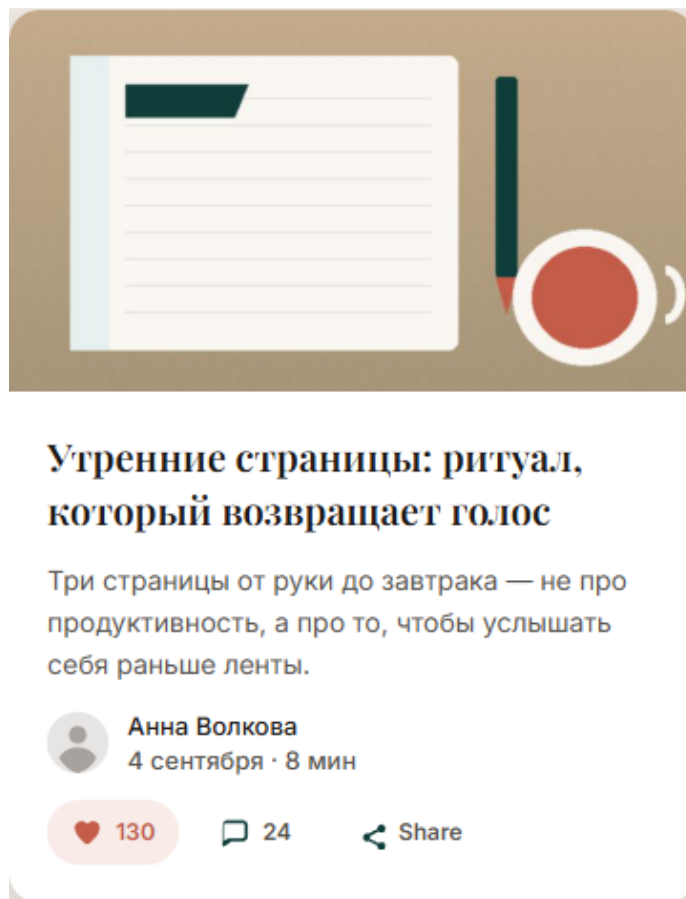


Рисунок 3.2 – Карточка статьи после увеличения счётчика лайков

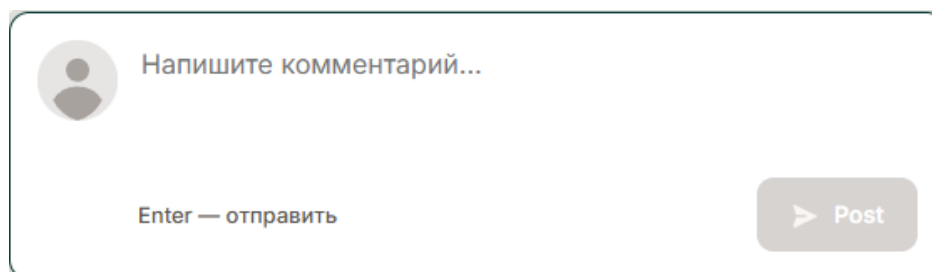


Рисунок 3.3 – CommentInput в фокусе, кнопка Post неактивна при пустом поле

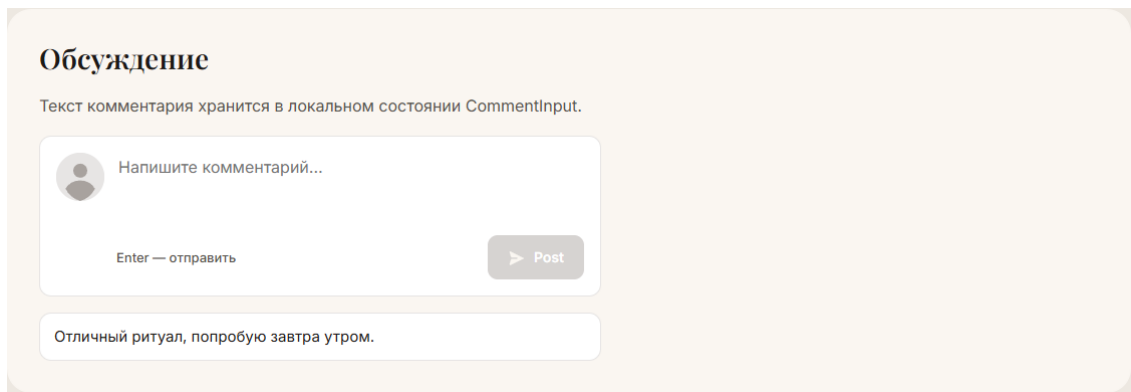


Рисунок 3.4 – Отправленный комментарий в локальном списке CommentInput

4 ЗАПУСК ПРОЕКТА

Исходный код размещён в *Git*-репозитории

<https://github.com/ed007angr/folio-blog-lab>

Инструкции по запуску продублированы в `README.md`. Кратко:

- 1 установить зависимости командой `npm install`;
- 2 запустить *dev*-сервер командой `npm run dev`;
- 3 открыть адрес, который выведет *Vite* (по умолчанию `http://localhost:5173`).

Проект собирается командой `npm run build` без ошибок. На работающем *dev*-сервере проверены клик по лайку (счётчик 128 → 130) и отправка комментария.

ВЫВОД

Реализованы функциональные *Vue*-компоненты блогговой платформы *Folio* на *JSX*: `LikeButton`, `CommentInput`, `ArticleCard`, а также вспомогательные `Avatar` и `PostButton`. Счётчик лайков и текст комментария хранятся в локальном состоянии через хук `useState`. Данные карточки статьи передаются через *props*. Классовые компоненты и внешние библиотеки состояния не использовались. Приложение запускается командой `npm run dev`.